Complying with the Right to Be Forgotten in Smart Mobility Data Sharing: A Delete-Only Redactable Consortium Blockchain

Tin Tironsakkul[1][a], Pradip Kumar Sharma[1][b], Deepak Puthal[2], Vinayagam Mariappan[3] and WonSik Hong[4]

[1]*Department of Computing Science, University of Aberdeen, Aberdeen, UK*

[2]*Department of Information System and Security, United Arab Emirates University, Abu Dhabi, UAE*

[3]*Mistral Solutions, Bengaluru, India*

[4]*SMR Automotive, Bucheon-si Gyeonggi-do, South Korea*

tin.tirons@gmail.com, pradip.sharma@abdn.ac.uk, deepak.puthal@ieee.org, vinayagam.m@mistralsolutions.com, Wonsik.hong@motherson.com

Keywords: Redactable Blockchain, Smart Mobility, Distributed Ledger, Consortium Blockchain

Abstract: The immutability of blockchain, while ideal as an alternative solution for decentralised data sharing and enhancing data authenticity in smart mobility, clashes fundamentally with growing demands for data mutability driven by privacy regulations. This tension threatens the practical adoption of blockchain in privacy-sensitive smart mobility ecosystems. To reconcile this conflict, we propose a novel data redaction approach designed for consortium blockchains. Our solution introduces a fine-grained, controlled, and accountable delete-only redaction function. The proposed protocol enables data subjects or trusted authorities to initiate the deletion of sensitive message data within transactions using a digital signature. The redaction request is subsequently validated and approved by a subset of trusted authorities via an accountable threshold signature scheme, ensuring transparency and non-repudiation. Performance analysis of the proof-of-concept implementation demonstrates minimal overhead in efficiency and scalability, which illustrates that our proposed protocol satisfies requirements for smart mobility applications.

1 Introduction

The development of the Internet of Things (IoT) and information technology has transformed the landscape of the mobility industry. Nowadays, many mobility-related devices, such as smart vehicles, traffic sensors, toll payment systems, and many others, are connected to share data with each other. The issues of data security and privacy have become a critical concern for smart mobility development due to the nature of personal data involved in smart mobility operations, which can include driver information, GPS logs, and travel activity.

Due to the single point of failure problem in centralised data storing and sharing that the mobility sector traditionally employs, the application of the consortium blockchain platform as a data-sharing solution for smart mobility has been investigated by various studies (Fan et al., 2023; Wei et al., 2024; Cui et al., 2024; Yadav et al., 2025). Additionally, it has also gained adoption to develop real-world smart city and vehicle projects such as the MOBI consortium blockchain for the smart mobility project (MOBI, 2025) and the Delphi project of the Horizon Europe initiative of the European Union for data security and collaboration in a unified transport system (DELPHI project, 2025).

Despite ongoing efforts to utilise consortium blockchains for smart mobility, a critical issue remains in the application of blockchain. The blockchain's immutability design, which prevents data tampering, conflicts with the right to be forgotten of the General Data Protection Regulation (GDPR). The regulation requires data controllers to remove personal data when no longer necessary or by consent withdrawal of the data subject without undue delay (Consulting, 2018). As much of the user data in smart mobility applications is under the definition of personal data in GDPR, the immutability attribute of blockchain will prevent data controllers from effi-

[a] https://orcid.org/0000-0002-8328-4955

[b] https://orcid.org/0000-0001-6620-9083

ciently complying with the regulation and removing any personal data within the blockchain once it is appended.

Despite existing research on redactable consortium blockchains for smart mobility (Lu et al., 2022; Zhou et al., 2024; Yang et al., 2025) aimed at addressing data management challenges, three critical issues persist in current schemes: **First**, state-of-the-art schemes utilise the chameleon hash primitive for the redaction protocol. This approach has weaknesses that allow compromised authorities to perform malicious data modification, and the trapdoor function incurs additional significant computational overhead for all mutable transactions that can substantially diminish the reliability and scalability of smart mobility's high-velocity network system. **Second**, recent fine-grained transaction- and message-level redaction schemes facilitate only one transaction per redaction, which can cause large communication and storage overhead in a growing blockchain. **Third**, the existing accountable threshold signature schemes induce a large overhead for storing a large signing quorum in blockchain, making them unsuitable for large-scale system authentication management in smart mobility.

Motivated by the aforementioned observations of the current research gaps and challenges, this work proposes a novel approach utilising a digital signature, meta-transaction, and a variant of the accountable threshold signature scheme called Accountable Threshold Signature with Indexed Quorum (*ATS-IQ*) to make redactable consortium blockchains suitable for smart mobility data sharing and comply with the GDPR. This combination enables the selective deletion of transaction message data with reduced overhead while ensuring accountability for all parties involved in issuing and approving the redaction.

2 Preliminaries

2.1 Blockchain Basics

Definition 1 (Blockchain) *A blockchain C is an array of blocks $C := [B_1, ..., B_n]$ where B_i is a block that contains transactions and i is the block index.*

A blockchain contains a sequence of blocks, with the first block (B_1) called the Genesis block. The total number of blocks in blockchain C is indicated as $len(C)$, and excluding the latest q blocks from a chain is expressed as $C^{\lceil q}$. For two chains from the same blockchain denoted as C_1 and C_2, if C_1 is the prefix of C_2 then we denote it as $C_1 \preceq C_2$.

Definition 2 (Block) *A block B is a data structure in the form $B := \langle bh, x \rangle$, where bh is the block header and $x := [TX_1, ..., TX_n]$ is the list of transactions.*

In common blockchains, a block header is a data structure that contains metadata related to the block. The block header can be defined in the form $bh := \langle mt, s, mr, ctr \rangle$, where $mt \in \mathbb{N}$ is the mining timestamp of the block, $s \in \{0,1\}^k$ is the hash of the previous block with $k \in \mathbb{N}$ being the security parameter, $mr \in \{0,1\}^k$ is the Merkle Root hash created from the hash of every transaction in the block, and ctr is the consensus mining proof. To generate a new block $B' := \langle bh' := \langle mt', s', mr', ctr' \rangle, x' \rangle$, the previous block hash s' is computed by $s' := H(bh := \langle mt, s, mr, ctr \rangle)$, where H is a cryptographic hash function which outputs in $\in \{0,1\}^k$.

Definition 3 (Transaction) *A transaction TX is a data structure in the form $TX := \langle z, d \rangle$ where $z \in \{0,1\}^k$ is the hash of the transaction computed by $z := H(d)$ and $d \in \{0,1\}^*$ is the message data.*

A transaction in a blockchain for smart mobility data sharing can include various data, such as traffic events, vehicle coordinate locations, driver personal information, smart contracts, and service data. For a transaction TX_i, we denote the index of the block in which it is mined as mb_i and the transaction order index in the block it is mined in as ti_i.

2.2 Blockchain Security Properties

There are three established fundamental security properties required for a well-developed and robust blockchain (Garay et al., 2015; Wu et al., 2025), which are chain growth, chain quality, and common prefix. For simplicity, we denote Γ as an immutable blockchain protocol and Γ^R as a modified redactable blockchain protocol.

2.2.1 Chain Growth

The chain growth property means that the number of blocks proportionally increases along with the time.

Definition 4 (Chain Growth) *Given two chains C_1, C_2 possessed by two honest nodes at time slots t_1, t_2, where $t_2 > t_1$. Then it holds that $len(C_2) - len(C_1) \geq \tau \cdot t$ for $t \in \mathbb{N}$ and $0 < \tau \leq 1$, where τ is the chain growth parameter, and $t = t_2 - t_1$.*

2.2.2 Chain Quality

The chain quality property informally means that the number of blocks manipulated by adversarial nodes' contribution in any length of a chain maintained by an honest node is bounded to less than a factor of μ.

Definition 5 (Chain Quality) *Given a portion of length ℓ-blocks possessed by an honest node at any*

time slot, for $\ell \in \mathbb{N}$. Then, the number of adversarial blocks in ℓ-blocks is at most μ, where $0 < \mu \leq 1$ is the chain quality parameter.

2.2.3 Common Prefix

The common prefix property informally means that if we take a blockchain at a different time slot from two honest nodes, the shorter chain is the prefix of the longer chain before exceeding the common prefix parameter k blocks.

Definition 6 (Common Prefix) *Given two chains C_1, C_2 possessed by two honest nodes at time slots t_1, t_2 where $t_2 > t_1$, then C_1 is the prefix of C_2. It can be denoted as $C_1^{\lceil q} \preceq C_2$, where $C_1^{\lceil q}$ is the chain obtained by removing the last q blocks from C_1 and $q \in \mathbb{N}$ is the common prefix parameter.*

As redactable blockchains cannot satisfy the common prefix property during redaction operations, previous studies introduce a new variation of the property, which is the redactable common prefix property (Shen et al., 2023; Li et al., 2023).

2.2.4 Redactable Common Prefix

The redactable common prefix property informally means that if there exists a block that violates the common prefix property, then there must also exist a block B_i and the valid modified block version B_i^R in different chains.

Definition 7 (Redactable Common Prefix) *Given two chains C_1, C_2 possessed by two honest nodes at time slots t_1, t_2 where $t_1 < t_2$ and such that either*

1. $C_1^{\lceil q} \preceq C_2$
2. *For any modified block $B_i^R \in C_1{}^{(t2-t1)+q}$ and $B_i \in C_2^q$, it satisfies $\Gamma^R.ValidateBlock(B_i^R) = 1$*

Where $C_1^{\lceil q}$ denotes the chain obtained by removing the last q blocks from C_1, and $q \in \mathbb{N}$ is the common prefix parameter.

2.3 Digital Signature

A digital signature (*DS*) scheme is a cryptographic primitive that generates public and private key pairs to generate a signature for any message.

Definition 8 (*DS*) *The digital signature scheme consists of a subset of algorithms $\{KeyGen, Sign, Verify\}$, which can be described as follows;*

- $(pk, sk) \leftarrow DS.KeyGen(\lambda)$, takes a security parameter (λ) and generates a public and private key pair (pk, sk).
- $\sigma \leftarrow DS.Sign(sk, m)$, takes sk and message (m) and generates a signature (σ).
- $\{0,1\} \leftarrow DS.Verify(pk, m, \sigma)$, takes a tuple of (pk, m, σ) and generates the verification output of the σ where the output 1 indicates that σ is a valid signature, while the output 0 indicates invalidity.

Definition 9 (*DS Correctness*) *The correctness of a digital signature requires that for all $(pk, sk) \leftarrow DS.KeyGen(\lambda)$, for all $\sigma \leftarrow DS.Sign(sk, m)$, we have that $DS.Verify(pk, m, \sigma) = 1$ is true.*

Definition 10 (*DS Security*) *We define the digital signature scheme security through the following game between a challenger and a probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ to forge the signature.*

- The challenger computes $(pk, sk) \leftarrow DS.KeyGen(\lambda)$, and sends the pk to $\mathcal{A}$.
- $\mathcal{A}$ issues signature queries with message m_i to the challenger who computes $\sigma_i \leftarrow DS.Sign(sk, m_i)$ and sends σ_i to $\mathcal{A}$.
- $\mathcal{A}$ gives a signature forgery pair (m, σ). If $m \neq m_i$ and $DS.Verify(pk, m, \sigma) = 1$, then $\mathcal{A}$ wins the game.

A digital signature scheme is proven to be existentially unforgeable against a chosen-message attack (EUF-CMA) where $\mathcal{A}$ has a negligible probability to forge a message/signature pair for any given message without having access to the private key.

2.4 Threshold Signature

A threshold signature (TS) scheme is a cryptographic primitive that generates a set of public and private keys for a group of n parties to enable δ private key holders to create a valid signature for a message.

Definition 11 (TS) *The threshold signature scheme consists of a subset of algorithms $\{KeyGen, Sign, Combine, Verify\}$, which can be described as follows;*

- $(PK, SK) \leftarrow TS.KeyGen(1^\lambda, n, \delta)$, takes a security parameter (λ), the number of signers n, and the threshold δ as inputs, to generate a public key set $PK := \langle pk_1, ..., pk_n \rangle$ and a private key set $SK := \langle sk_1, ..., sk_n \rangle$.
- $\sigma_i \leftarrow TS.Sign(sk_i, m)$, takes sk_i and message (m) and generates a signature set $\sigma_i := \langle R_i, z_i \rangle$.
- $\sigma_m \leftarrow TS.Combine(PK, m, S, \{\sigma_i\}_{i \in S})$, takes public key set PK, message m, signer set S, signature set σ_i to output TS signature $\sigma_m := \langle R, z \rangle$.
- $\{0,1\} \leftarrow TS.Verify(PK, m, \sigma_m)$, takes a tuple of (PK, m, σ_m) and generates the verification output of σ_m where the output 1 indicates that σ_m is a valid TS signature, while 0 indicates invalidity.

Definition 12 (*TS Correctness*) *The correctness of a threshold signature requires that for all* $(PK, SK) \leftarrow TS.KeyGen(1^{\lambda}, n, \delta)$*, for all* $\sigma_i \leftarrow TS.Sign(sk_i, m)$*, for all* $\sigma_m \leftarrow TS.Combine(PK, m, S, \{\sigma_i\}_{i \in S})$ *where* $len(S) \geq \delta$*, we have* $TS.Verify(PK, m, \sigma_m) = 1$ *is true.*

Definition 13 (*TS Security*) *We define the threshold signature scheme security through the following game between a challenger and* $\mathcal{A}$ *to forge the signature.*

- The challengers n computes $(PK, SK) \leftarrow TS.KeyGen(1^{\lambda}, n, \delta)$, and sends the PK to $\mathcal{A}$.
- $\mathcal{A}$ issues queries with message m_a to a δ subset (S) of n who compute $\sigma_i \leftarrow TS.Sign(sk, m_a)$, $\sigma_{m_a} \leftarrow TS.Combine(PK, m_a, S, \{\sigma_i\}_{i \in S})$ and send σ_{m_a} to $\mathcal{A}$.
- $\mathcal{A}$ gives a signature forgery pair (m, σ_m). If $m \neq m_{m_a}$ and $TS.Verify(PK, m, \sigma_m) = 1$, then $\mathcal{A}$ wins the game.

A threshold signature scheme is proven to be existentially unforgeable against EUF-CMA, where $\mathcal{A}$ has a negligible probability of forging a signature pair for any given message without having access to at least δ private keys.

2.5 Accountable Threshold Signature

An accountable threshold signature (ATS) scheme is an extension of the threshold signature scheme that adds the *Trace* algorithm to provide accountability to the signing process in the form $S \leftarrow ATS.Trace(PK, m, \sigma_m)$, which takes a tuple of public key set PK, message m, and signature σ_m to reveal the quorum of signers S that generate the signature σ_m.

Definition 14 (*ATS Trace Correctness*) *The trace correctness of an accountable threshold signature requires that in addition to the TS Correctness, we have that* $ATS.Trace(PK, m, \sigma_m) = S$ *is true.*

3 Design Challenge and Goals

In addition to satisfying the security properties described in Section 2.2, our aim is to achieve the corresponding functional and security design goals for the redactable blockchain protocol in smart mobility applications, which we also use to evaluate existing redaction approaches;

G1 (**Transparency**) As mentioned by related works in Section 1, transparency is one of the fundamental aspects of blockchain technology that is essential for secure data sharing and storage in smart mobility applications. By recording all data in the blockchain, the blockchain transparency attribute allows the authorities to efficiently verify and audit the data integrity compared to traditional centralised systems. Therefore, the protocol should not exclude data from the blockchain to make it mutable.

G2 (**Fine-Grained Control**) A centralised redaction control can create a single point of failure, where a compromise can lead to a total system failure (Li et al., 2024). To achieve fine-grained control, the redaction protocol should be performed at a message level to preserve the blockchain transaction history. In an ideal scenario that satisfies the GDPR requirement, data subjects and/or data controllers should be able to directly issue the redaction order for personal data to blockchain nodes using a valid redaction permission proof.

G3 (**Controlled Delete-only Redaction**) The implementation of mutability in the blockchain can create security vulnerabilities, allowing adversarial entities to modify and inject malicious data that can cause real-world damage. Therefore, a redaction implementation should operate via a secure and controlled procedure that completely prevents arbitrary data rewriting.

G4 (**Accountability**) As the redacting operation can compromise the security and integrity of the blockchain, a secure redaction protocol should include accountability and traceability mechanisms that allow observers to distinguish whether the data in transactions is original or redacted while also help validate and indicate responsible entities to deter abuses (Ye et al., 2023; Zhang et al., 2024).

4 Existing Redaction Approaches

We classify the relevant approaches based on their methods of operation into four categories, which are off-chain database (Section 4.1), chameleon hash (Section 4.2), meta-transaction (Section 4.3), and digital signature approaches (Section 4.4).

4.1 Off-chain Database

The off-chain database approach is based on the already existing cryptocurrency Layer 2 protocols, such as Bitcoin's Lightning Network (LN), Ethereum's Polygon, and Hyperledger Fabric Purge. Generally, the principal mechanism of this approach is to store specific data outside of the blockchain while keeping the data hash on the blockchain for indexing and verification, thus making the data mutable regardless of the immutability of the blockchain (Al-Zaben et al.,

Approach	Trans-parency (G1)	Fine-Grained Control (G2)	Redaction Level (G2)	Controlled Delete-only (G3)	Account-ability (G4)
Off-chain	×	P	Message	✓	×
Chameleon Hash	✓	✓	Message	P	✓
Meta-TX	✓	P	Message	P	P
DS	✓	✓	Block	P	✓
DS and Meta-TX	✓	✓	Transaction	P	✓
Our, *DS*, *ATS* and Meta-TX	✓	✓	Multi-Message	✓	✓

Table 1: Comparison Overview With Other Redaction Approaches
The letter P denotes partial achievement.

2018).

Although this approach allows blockchain to preserve its immutability while complying with GDPR, the mutable and redacted data being external to the blockchain reduces its transparency (G1) and accountability (G4) (Politou et al., 2021).

4.2 Chameleon Hash

Chameleon hash is the blockchain redaction approach first proposed by Ateniese et al. (Ateniese et al., 2017) and has been improved by various studies since then to improve its security, efficiency, accountability, and applicability (Ma et al., 2022; Jia et al., 2022; Duan et al., 2023; Li et al., 2023). The concept of this approach is to create a trapdoor collision hash function that allows the trapdoor key holders to modify data while maintaining the same hash.

The Chameleon hash method is widely considered state-of-the-art, as it is the current most investigated and adapted blockchain redaction primitive. The main advantage of this approach is that it allows flexibility in modifying the blockchain content. While the Chameleon hash approach can achieve a negligible probability of external PPT adversaries, the effect of security failure from compromised secret key holder(s) is unrestricted data tampering. As such, this flexibility advantage is also a weakness in that entities with trapdoor keys can uncontrollably modify mutable data, allowing malicious entities to inject malicious data into the blockchain.

Many studies proposed partial solutions to partially address this issue, where some introduced a monetary cost or penalty system (Chen and Gao, 2022; Ou and Ye, 2023), an accountability system (Duan et al., 2023; Shao et al., 2023), and redaction voting between shared key holders (Ateniese et al., 2017; Ma et al., 2022; Jia et al., 2022). However, these mechanisms can deter but do not completely prevent malicious tampering, and no chameleon hash-based approach proposed by previous studies has been able to achieve controlled delete-only redaction where no arbitrary data editing is permissible (G3) to the best of our knowledge.

4.3 Meta-transaction (Meta-TX)

The meta-transaction redaction approach operates by using a special type of transaction (meta-transactions) to issue and record the redaction process.

Two distinguished studies proposed a framework for the meta-transactions approach that applies to consortium blockchains. The first work is μchain by Puddu, Dmitrienko, and Capkun (Puddu et al., 2017), which creates active and inactive states of mutable transactions that can be changed with a meta-transaction. However, this proposal only replaces the active data with its inactive encrypted version, which can still be accessed with a decryption key. Thus, the approach only partially achieves the controlled delete-only goal (G3).

The second work is Memory Optimised and Flexible BC (MOF-BC) by Dorri, Kanhere, and Jurdak (Dorri et al., 2019), which proposes the blockchain protocol to generate transaction hashes without using the content of the transaction message data so that data redaction does not affect the consistency of the hash link. The approach allows only centralised figures to initiate the redaction process (G2).

4.4 Digital Signature (*DS*)

Few studies employed the digital signature scheme as a redaction method by providing proof of redaction permission. There are three prominent studies that utilise this approach.

The first work is by Cai, Ren, and Zhang (Cai et al., 2020), which proposes using a traceable threshold ring signature scheme that allows transaction senders to obscure their identity and transaction content. The sender can issue a delete-only redaction for their data by revealing one of the two aforementioned pieces of information to full nodes. The redaction protocol proposed by this work operates on block-level modification (G3) and does not provide accountability for the TAs (Trusted Authorities) responsible for the redaction approval vote (G4).

The second work is by Li and Ma (Li and Ma, 2021), which modifies the block header structure

by adding a redaction signature field. The protocol allows block hash consistency to be broken if the modified blocks include a trusted authority's redaction signature. The protocol demonstrates high efficiency and scalability but suffers from the weakness of centralised redaction permission (G2) and arbitrary block-level modification (G3).

The third work is by Heo, Ramachandran, and Jurdak (Heo et al., 2024) that proposes combining the digital signature and meta-transaction approach. The study employs the one-time cryptographic key generation scheme and the zero-knowledge succinct non-interactive arguments of knowledge (zk-SNARK) primitive that allow redaction to be anonymously performed. While the protocol in this work does not require centralised trust authorities, it allows unlimited arbitrary transaction-level modification (G3). Additionally, the redaction anonymity feature provided by this work is unsuitable for smart mobility applications that require accountability (G4).

5 Our Proposed Protocol

In this section, we describe our proposal. From this section forward, we denote our modified redactable blockchain protocol as Γ^{R}_{δ}, where $\delta \in \mathbb{N}$ indicates the parameter of the accountable threshold signature requirement for redaction approval by TAs.

5.1 Accountable Threshold Signature with Indexed Quorum

We construct the Accountable Threshold Signature with Indexed Quorum (*ATS-IQ*) based on Schnorr's public ATS scheme (Micali et al., 2001). This scheme stores and processes the signing quorum's public keys' index (*Si*) for better scalability on a large threshold signature. The scheme modifies the $ATS.Trace$ algorithm into *ATS-IQ.ITrace* as follows;

$\{0,1\} \leftarrow$ *ATS-IQ.ITrace*$(PK, m, \sigma_m := \langle R, z\rangle, Si)$, validators input the group's public key set $PK := \langle pk_1, ..., pk_n\rangle$, message m, σ_m, and a tuple containing the index of each signing quorum's public key *Si* to verify that the σ_m signature is signed by S. The algorithm first matches the index *Si* to create a public key set of the signing quorum S before following the same procedure as the Schnorr $ATS.Trace$ algorithm in (Micali et al., 2001).

5.2 System Entities and Components

We classify entities based on their interaction with the blockchain system in our design into three types:

Trusted Authorities (TAs) refer to consortium blockchain owners or trusted third parties who are responsible for initialising the system, maintaining access control for participants to the blockchain, participating as full nodes, approving redaction, and enforcing blockchain redaction policy via blockchain auditing. TAs should be composed of collaborating government agencies with a stake in smart mobility systems to ensure the security of our model in practice.

Registration Authorities (RAs) refer to a group of entities responsible for registering participants into the blockchain system. RAs will jointly handle their operation through mutual consensus and audits. However, RAs will not participate in subsequent redaction or other blockchain processes.

Full Nodes refer to blockchain moderators who are responsible for blockchain maintenance, transaction mining, and data redacting. In smart mobility applications, full nodes can be service providers, TAs, and RSUs that act as intermediaries for users' access to the blockchain data by serving as parent nodes to users' IoT lightweight node devices.

Senders (sd) refer to participants with valid access to generate and broadcast transactions to full nodes for mining into the blockchain. In smart mobility applications, senders can be the data subjects or data controllers who own related data.

Our design for redaction functions includes three critical components:

Redactable Consortium Blockchain is a modified consortium blockchain with redaction functions that contains smart mobility data in the form of transactions within each block.

Access Control List (*ACL*) is a data structure in a consortium blockchain system that outlines the participants and their network access permissions. This list contains the public keys of all entities involved, which are then used to validate permissions for on-chain interactions, including data access and transaction submission.

Unconfirmed Redaction List (*URL*) is local data storage for new or reconnected full nodes to temporarily store the block and transaction index of unconfirmed redaction transactions in a block that has not yet been downloaded.

5.3 Modified Transaction Structure

In the conventional immutable blockchain Γ protocol, the hash value (s) of a block is generated from the block header, which includes the Merkle Root of the transactions to make transactions immutable, as described in Section 2.1. To achieve message-level redaction that removes only required data and pre-

serves transaction history, our proposal introduces a new transaction structure as follows;

Definition 15 (Γ^R_δ Modified Transaction) A modified transaction in the Γ^R_δ protocol is defined as the form $\Gamma^R_\delta.TX := \langle z, h, d \rangle$ where $z \in \{0,1\}^k$ is the hash of the transaction, h is the transaction header, and $d \in \{0,1\}^*$ is the message data in the transaction.

We modify the transaction structure by adding one additional component, namely the transaction header (h). A transaction header is a set of immutable transaction metadata that serves as an essential component to create a hash of the transaction. Our proposed Γ^R_δ protocol classifies transactions into transactions (TX) and redaction transactions (RTX).

Transaction (TX) is a transaction that behaves as any immutable transaction in the Γ protocol, except for the addition of the transaction header and the change in the transaction generation and validation process, where the transaction hash z is computed by $TX.z := H(h)$, which allows data redaction without affecting the hash consistency. This type of transaction contains two values in transaction headers;

- Transaction Script (ts) is a script in the form $ts := \langle pk_{sd}, \sigma \rangle$ where σ is computed from $\sigma \leftarrow DS.Sign(sk_{sd}, m)$ where $m := h \setminus ts$.
- Message Data Hash (q) is the hash of the original message data (d) computed from $q := H(d)$ for securing the integrity of the data and serve as a redaction indication.

Redaction Transaction (RTX) is a meta-transaction to issue redaction for the message data in the specified transactions in ri. The transaction hash z is computed by $RTX.z := H(h)$. Unlike TX, these redaction transactions do not contain transaction data ($d = \varnothing$) and the transaction hash is generated by TAs after signing and combining σ_m. This type of transaction contains three values in transaction headers;

- Redact Index (ri) is an index data $ri := \langle mb_{mtx_1}, ti_{mtx_1}, ..., mb_{mtx_n}, ti_{mtx_n} \rangle$, where mb and ti are block and transaction order index of each targeted TX (mtx) for redaction.
- Redaction Script (rs) is a script $rs := \langle pk_{rq}, \sigma \rangle$ where σ is computed from $\sigma \leftarrow DS.Sign(sk_{rq}, ri)$, rq is the redaction requester who can be the sender (sd) or a TA.
- TAs Approval (taa) is a data $taa := \langle \sigma_m, e, Si \rangle$ where $e := H(PK)$ is the hash of TAs' *ATS-IQ* public key set, σ_m is a signature signed by a S subset of TAs computed from $\sigma_i \leftarrow ATS\text{-}IQ.Sign(sk_i, m)$, $\sigma_m \leftarrow ATS\text{-}IQ.Combine(PK, m, S, \{\sigma_i\}_{i \in S})$ where $len(S) \geq \delta$ and $m := h \setminus taa$.

5.4 Threat Model

In our threat model for a consortium blockchain setting, we consider an **internal** or **external** PPT adversary $\mathcal{A}$ may attempt to initiate an unauthorised redaction order or manipulate the mutability mechanism to inject malicious data into the blockchain. Adversary $\mathcal{A}$ can be compromised/malicious senders, full nodes, or TAs. As such, we assume the adversary's capabilities from a practical perspective are as follows.

- $\mathcal{A}$ possesses polynomial-time computational capabilities, which means that they cannot break the underlying cryptographic primitives.
- $\mathcal{A}$ can attempt to forge transactions, manipulate captured transaction data, or replace signed signatures with forged ones. The adversary can also be compromised senders that aim to exploit the redaction function to inject malicious data or remove evidence of malicious activities.
- $\mathcal{A}$ can control a number of compromised TAs, but cannot exceed more than the majority threshold ($\geq \delta$) of existing TAs in the system, which is a commonly used security assumption of a blockchain system.

We assume that RAs are only involved in the cryptographic key registration and not in the subsequent blockchain operations. While future key registration might be affected if RAs become compromised afterwards, the blockchain operation and previously mined transactions would remain secure, and the potential impact of compromised RAs does not result in a single point failure of the blockchain system.

5.5 Redactable Blockchain Protocol

In this section, to clearly describe the Γ^R_δ blockchain redaction scheme, we introduce each step in detail as follows; SETUP, APPEND, VALIDATE, and DELETE. We denote $\mathcal{ADD} := \{0,1\}$ as the parameter that indicates a function is being executed during the APPEND operation to receive new transactions or the latest candidate block.

SETUP is the operation where TAs set up the blockchain system, RAs register cryptographic keys, and new or rejoined nodes update their blockchain.

1. Initialisation. At the start of the process, TAs initialise the Γ^R_δ blockchain system with Γ security parameters and δ TAs threshold redaction approval policy parameter. Suppose there are n TAs in the system. Based on the threat model, there are various majority security models that TAs can adopt, such as standard blockchain $51\% = n/2+$

1, Byzantine fault tolerant $2/3$, or higher for a secure redaction policy. The tradeoff is that a higher threshold provides better security but increases computational and storage overhead, which we explore later in Section 7.3. Then, TAs compute $(PK, SK) \leftarrow \textit{ATS-IQ.KeyGen}(1^{\lambda}, n, \delta)$ and register with RAs. Each generation of the new *ATS-IQ* key to add or remove TAs is recorded in ACL along with the hash of the public key set (e) for validation, and only the latest generated PK can be used in new redaction operations.

2. Registration. Each new participant computes $(pk, sk) \leftarrow \textit{DS.KeyGen}(\lambda)$ and registers the public-private key pair to RAs.
3. Updating. Full nodes join the network and update their blockchain to the longest chain state. Full nodes perform the $\Gamma^R_\delta.\textit{ValidateBlock}(C, B_i, ACL, 0, URL)$ (Algorithm 1) for each received block B_i.

APPEND is the operation of generating, broadcasting, and mining new TXs into the new block. The operation workflow for each step is as follows;

1. TX Generation. Sender (sd) generates a TX, and runs $\sigma \leftarrow \textit{DS.Sign}(sk_{sd}, h \setminus ts)$ to sign the transaction signature in ts.
2. TX Broadcasting. The sender broadcasts the transaction to full nodes who validate it with $\Gamma^R_\delta.\textit{ValidateTX}(TX)$ (Algorithm 2). If the validation is successful, full nodes propagate the transaction to other nodes.
3. TX Mining. A full node mines the TX into the newest block B', adds B' to its local blockchain, and propagates B' to other nodes that run $\Gamma^R_\delta.\textit{ValidateBlock}(C, B', ACL, 1, URL)$ (Algorithm 1) before adding B' into their own blockchain.

VALIDATE is the data validation operation that occurs during the broadcasting/mining (APPEND), redacting (DELETE), or auditing process. The operation is necessary to ensure the integrity of the blockchain, enforce the redaction policy, and verify the legitimacy of the transaction and block. Note that the chain validation function in Γ^R_δ follows the same procedure as the Γ protocol. Moreover, the modified validation processes described below follow the predefined Γ protocol procedure set up by the TAs, including the validation of data content and format.

$\Gamma^R_\delta.\textit{ValidateBlock}$ is the process for validating a block, as shown in Algorithm 1. The algorithm first follows the Γ protocol to check the block's validity. Next, the process checks if there are unconfirmed redaction transactions that must exist in the block. Afterwards, the algorithm employs an appropriate Γ^R_δ transaction validation process for each transaction inside the block based on its type.

input : $C, B_i := \langle bh_i, x_i \rangle, ACL, \mathcal{ADD} := \{0,1\}, URL$
output: $\{0,1\}$
if $i \in URL$ **then**
  for $ti \in URL$ **do**
    if $ti \notin x_i$ **then**
      return 0
    end
  end
end
for $tx := \langle z, h, d \rangle \in x_i$ **do**
  if $rs \in h$ **then**
    if $\Gamma^R_\delta.\textit{ValidateRTX}(C, tx, ACL, \mathcal{ADD}, \varnothing) = 0$ **then**
      return 0 ;
    end
    if $ti_{tx} \in URL$ **then**
      REMOVE $URL \leftarrow \langle mb_{tx}, ti_{tx} \rangle$;
    end
  end
  else
    if $\Gamma^R_\delta.\textit{ValidateTX}(C, tx) = 0$ **then**
      return 0 ;
    end
  end
end
return 1

Algorithm 1: $\Gamma^R_\delta.\textit{ValidateBlock}$

$\Gamma^R_\delta.\textit{ValidateTX}$ is the process for validating a TX. As shown in Algorithm 2, the process starts by validating that the transaction hash value matches the hash of the transaction header. Then, the process validates the hash of transaction data with the q value in the header to check if the data has been modified. If the current data hash does not match the q value, the process will look up the redaction transaction (RTX) and validate it with $\Gamma^R_\delta.\textit{ValidateRTX}(C, RTX, ACL, 0, TX)$ (Algorithm 3). If the redaction transaction is not found in the blockchain, the process will check whether the block exists using the length of the chain. If the length is lower than the block index, the process will add the index of the unconfirmed redaction transaction into URL for later checking, as shown in Algorithm 1. If the current data hash matches the q value, the process

will validate the public key and signature in ts.

input : $C := \langle B_1, ..., B_N \rangle, TX := \langle z, h := \langle ts, q \rangle, d \rangle, URL$
output: $\{0,1\}$
if $H(h) \neq z$ **then**
  return 0
end
if $H(d) \neq q$ **then**
  if $len(C) < d.mb_{rtx}$ **then**
    ADD $URL \leftarrow \langle d.mb_{rtx}, d.ti_{rtx} \rangle$
  end
  else if $d.ti_{rtx} \in B_{d.mb_{rtx}}$ **then**
    $RTX \leftarrow B_{d.mb_{rtx}}[d.ti_{rtx}]$;
    if $\Gamma^R_\delta.ValidateRTX(C, RTX, \varnothing, 0, TX) \neq 2$ **then**
      return 0
    end
  end
  else
    return 0
  end
end
else if $DS.Verify(pk_{sd}, h \setminus ts, \sigma) = 0$ **then**
  return 0
end
return 1 ;

Algorithm 2: $\Gamma^R_\delta.ValidateTX$

$\Gamma^R_\delta.ValidateRTX$ is the process for validating an RTX. The function for this process is shown in Algorithm 3, where it first validates the rs signature and confirms that d does not contain any data ($\varnothing$). If the RTX is approved by the TAs, the process will check the transaction hash against the transaction header and validate the signature in taa. Next, the function searches for the specified TXs with the provided ri data. If a TX is assigned in MTX, the function will search for only that TX. For the APPEND operation ($\mathcal{ADD} = 1$), the function will confirm that the TX is not yet redacted to prevent redaction duplication and that the rs is signed by either the TX sender or a TA. For other operations ($\mathcal{ADD} = 0$), if TXs are not redacted, the process will return invalid when it is performed during blockchain auditing by TAs or execute the redaction function (Algorithm 4) for the RTX with a valid taa signature during blockchain updating by full nodes. If the validation passes all previous checks, the function will return 1, which denotes the RTX is valid but unapproved, or 2, which indicates the RTX is approved and ready to be mined.

DELETE For TXs, their sender or a TA can issue

input : $C := \langle B_1, ..., B_N \rangle, RTX := \langle z', h' := \langle ri := \langle mb_{mtx_1}, ti_{mtx_1}, ..., mb_{mtx_n}, ti_{mtx_n} \rangle, rs := \langle pk_{rq}\sigma \rangle, taa := \{\varnothing, \langle \sigma_m, e, Si \rangle\} \rangle, d' \rangle, ACL, \mathcal{ADD} := \{0,1\}, MTX := \{\varnothing, TX\}$
output: $\{0,1,2\}$
$PK \leftarrow ACl.PKs[e]$;
if $DS.Verify(pk_{rq}, ri, \sigma) = 0 \vee d' \neq \varnothing \vee (taa \neq \varnothing \wedge (H(h') \neq z' \vee ATS\text{-}IQ.Verify(PK, h' \setminus taa, \sigma_m) = 0 \vee ATS\text{-}IQ.ITrace(PK, h' \setminus taa, \sigma_m, Si) = 0 \vee (\mathcal{ADD} = 1 \wedge PK \text{ is not the latest generated})))$;
then
  return 0
end
for $mb_{mtx}, ti_{mtx} \in ri$ **do**
  $TX := \langle z, h := \langle ts, q \rangle, d \rangle \leftarrow B_{d.mb_{mtx}}[ti_{mtx}]$
  if $MTX = \varnothing \vee TX = MTX$ **then**
    if $\mathcal{ADD} = 1$ **then**
      if $H(d) \neq q \vee (pk_{rq} \neq pk_{sd} \wedge pk_{rq} \notin ACL.TA)$ **then**
        return 0
      end
    end
    else if $H(d) = q$ **then**
      if *Performs in VALIDATE* **then**
        return 0
      end
    end
  end
end
if *Performs in SETUP* **then**
  if $taa = \varnothing$ **then**
    return 0
  end
  $\Gamma^R_\delta.RedactTX(C, RTX)$
end
if $taa = \varnothing$ **then**
  return 1
end
return 2

Algorithm 3: $\Gamma^R_\delta.ValidateRTX$

an order for the message data to be removed from the blockchain. The operation workflow is as follows;

1. RTX Generation. The sender of the TXs or a TA can create an RTX to redact message data in TXs by generating the RTX transaction and executing $\sigma \leftarrow DS.Sign(sk_{rq})$ to sign the rs.

2. RTX Broadcasting. The TXs sender, or TA, broadcasts the RTX to full nodes, who validate the transaction with $\Gamma^R_\delta.ValidateRTX(C,RTX,ACL,1,\varnothing) = 1$ and then propagate the transactions to other nodes.
3. RTX Approving. δ TAs validate the RTX with $\Gamma^R_\delta.ValidateRTX(C,RTX,ACL,1,\varnothing)$ and validate that each TX assigned in *ri* is applicable to be redacted according to regulations or transaction dependencies. If there is any invalidity in the request, TAs can reject the RTX and send the details so the requester can revise the request. If the RTX is considered valid, the TAs will execute $\sigma_i \leftarrow$ *ATS-IQ.Sign*$(sk_i, h \setminus taa)$, $taa := \langle \sigma_m, e, Si \rangle \leftarrow$ *ATS-IQ.Combine*$(PK, h \setminus taa, S, \{\sigma_i\}_{i \in S})$ to approve the transaction. Lastly, TAs generate the transaction hash for the RTX and propagate the approved RTX.
4. RTX Mining. A full node verifies the RTX with $\Gamma^R_\delta.ValidateRTX(C,RTX,ACL,1,\varnothing) = 2$ and ensures that the *PK* hash (e) being used is the latest generated key. Then, they mine it into the newest block B'', add B'' to its local blockchain, and propagate B'' to other nodes that run $\Gamma^R_\delta.ValidateBlock(C,B'',ACL,1,\ URL) = 1$ and add B'' to their own blockchain.
5. TX Redaction. After adding the block to their blockchain, each full node executes the $\Gamma^R_\delta.RedactTX(C,RTX)$ shown in Algorithm 4 to replace the message data in each TX assigned in *ri* with the block and transaction index of the RTX.

input : $C := \langle B_1, ..., B_N \rangle, RTX := \langle z', h' := \langle ri', rs', taa' \rangle, d' \rangle$
output: $\{0,1\}$
for $mb_{mtx}, ti_{mtx} \in ri'$ **do**
  if $ti_{mtx} \notin B_{mb_{mtx}}$ **then**
    return 0
  end
  $TX := \langle z,h,d \rangle \leftarrow B_{mb_{mtx}}[ti_{mtx}]$;
  $d \leftarrow \langle mb_{RTX}, ti_{RTX} \rangle$;
end
return 1

Algorithm 4: $\Gamma^R_\delta.RedactTX$

6 Security and Design Goal Analysis

In this section, we perform a security analysis of the proposed redactable blockchain protocol. We also discuss mitigation of potential failure modes and how our designed protocol achieves the design goal requirement formulated in Section 3.

6.1 ATS-IQ Security

Lemma 1 The Accountable Threshold Signature with Indexed Quorum (*ATS-IQ*) is secure/unforgeable and accountable if the underlying ATS is secure.

As a Schnorr-based accountable threshold signature scheme, the *ATS-IQ* should also satisfy the notion of unforgeability against EUF-CMA while being accountable, since our scheme does not add or modify the underlying algorithms beyond that of $ATS.ITrace$ in the accountable threshold scheme. Hence, we directly transfer the results of the security analysis in (Micali et al., 2001; Boneh and Komlo, 2022) regarding the security of Schnorr-based *ATS-IQ*, giving the conclusions in the above lemma.

6.2 Message Redaction Legitimacy

The proposed redactable consortium blockchain employs a two-layered security mechanism to govern the redaction process. The first layer ensures the integrity and authenticity of redaction orders through the use of robust digital signatures. The second layer secures the redaction authorisation process via the approvals issued by TAs using the *ATS-IQ* scheme.

Theorem 1 (Redaction Security) *If the signature scheme used for redaction transaction signing and ATS-IQ for redaction approval are EUF-CMA secure, then $\mathcal{A}$ has a negligible probability of executing illegal redaction by forging digital signatures without having access to the private key and ATS-IQ signatures without having access to at least t private keys.*

Proof. Suppose that there exists a PPT adversary $\mathcal{A}$ against the redaction security, $\mathcal{A}$ can generate a valid redaction transaction $RTX := \langle z, h := \langle ri, rs := \langle pk_{rq}\sigma \rangle, taa := \langle \sigma_m, e, Si \rangle \rangle, d \rangle$ at the challenge phase if they succeed in the following two events:

E1 $\Gamma^R_\delta.ValidateRTX(C,RTX,ACL,1,\varnothing) = 1$
 - $DS.Verify(pk_{rq}, ri, \sigma) = 1$

E2 $\Gamma^R_\delta.ValidateRTX(C,RTX,ACL,1,\varnothing) = 2$
 - *ATS-IQ.Verify*$(PK, h \setminus taa, \sigma_m) = 1$
 - *ATS-IQ.ITrace*$(PK, h \setminus taa, \sigma_m, Si) = 1$

We consider two security compromise scenarios based on the adversary capability assumption described in Section 5.4.

6.2.1 Only external adversary

Assume that $\mathcal{A}$ attempts to win the digital signature security game to forge the signature pair (pk_{rq}, σ) without having access to the challenger's private key. The chance of this forgery game scenario succeeding is negligible according to the established EUF-CMA security definition.

6.2.2 An internal sender and/or a subset of TA quorum (up to δ - 1) are compromised

Suppose that $\mathcal{A}$ (as a compromised/malicious sender or TA) possesses a valid private key to generate and execute the RTX, therefore succeeding in the event E1. The RTX will then need to be validated and approved by δ TAs. In this scenario, $\mathcal{A}$ then attempts to win the *ATS-IQ* security game to forge the signature pairs $(\{pk_i\}_{i \in S}, \sigma_m)$. $\mathcal{A}$ has negligible probability of succeeding without having access to δ private keys (Boneh and Komlo, 2022).

□

6.3 Blockchain Security Properties

As we build our redactable blockchain protocol Γ^R_δ upon the immutable Γ protocol, the basic intuition for our security analysis of the three immutable blockchain security properties is that, given Γ satisfies the three properties, our Γ^R_δ will also satisfy the same properties (where the common prefix property is a redactable variant). We discuss how our Γ^R_δ protocol meets the requirements as follows;

Theorem 2 (Chain Growth) *Assume Γ satisfies (τ, t)-chain growth, then Γ^R_δ satisfies (τ, t)-chain growth.*

Proof. As Γ^R_δ removes only message data in a transaction (message-level), and it is not possible to remove a block or transaction from the chain, the length of the blockchain ($len(C)$) is unaffected at any time slot. Hence, the redaction operation will not damage the chain growth property. □

Theorem 3 (Chain Quality) *Assume Γ satisfies (μ, ℓ)-chain quality, then Γ^R_δ satisfies (μ, ℓ)-chain quality.*

Proof. As Γ^R_δ is built upon Γ to implement the data redaction function for transactions, Γ^R_δ will behave in the same manner as Γ if no redaction has occurred. Since the redaction operation in Γ^R_δ allows only "deletion" of the data and not "modification" into any arbitrary data, $\mathcal{A}$ will not be able to inject malicious data into the blockchain via data redaction operations. As such, Γ^R_δ satisfies this security property. □

Common Prefix Γ^R_δ satisfies this security property only in the scenario that no redaction has taken place or after all nodes complete the redaction, similar to that of the immutable blockchain protocol Γ. This property is violated when any redaction occurs, as with other redaction approaches.

Theorem 4 (Redactable Common Prefix) *Assume Γ satisfies k-common prefix, then Γ^R_δ satisfies k-redactable common prefix.*

Proof. Given that no redaction occurred, or after all nodes complete redaction operations, Γ^R_δ behaves as the Γ. In the case of a redaction, assume an adversary $\mathcal{A}$ broadcasts redaction transactions to redact message data of transactions in block B_i, an honest node U_1 accepts the transactions and proposes a candidate block B_j, mines B_j into their chain C_1, and applies the redaction to block B^R_i. The candidate block B_j is later accepted by an honest node U_2 into their chain C_2 at time slot t_2. The unforgeability of the digital signature scheme ensures that $\mathcal{A}$ is not able to forge valid signatures for redaction transactions in B_j efficiently. Hence, with U_1 and U_2 being honest nodes and accepting the redaction in B_j, it must be in the case that $\Gamma^R_\delta.ValidateBlock(C_1, B^R_i, ACL, 0, URL) = 1$ and $\Gamma^R_\delta.ValidateBlock(C_2, B_j, ACL, \{0,1\}, URL) = 1$.

□

6.4 Failure modes, mitigation, and design goals

There are two modes of failure that we consider to be related to the redaction security of our protocol;

- **Partial ($< \delta$) compromised quorum** A sub-threshold number of compromised TAs is insufficient to allow for unauthorised redaction. However, the compromised TAs can still degrade the efficiency of the redaction process by refusing to participate in the honest *ATS-IQ* signing protocol.
- **Majority ($\geq \delta$) compromised TAs** Should $\mathcal{A}$ successfully compromise δ or more TAs, $\mathcal{A}$ can achieve both events E1 and E2, thereby enabling malicious redaction.

The four design goals described in Section 3 serve as layers of mitigation for the above failure modes.

6.4.1 G1 (Transparency)

Our approach allows the direct blockchain storage of all data types, including personal data, negating the need for an external database. This method is instrumental for data governance, as it cryptographically preserves a complete history of transactions and

redactions. The immutability of these records provides verifiable evidence during data audits, serving as a robust defence against malicious data tampering. These on-chain redaction records also enable rigorous internal auditing to confirm the integrity and compliance of all TAs contributing to the redaction process.

6.4.2 G2 (Fine-Grained Control)

The proposed approach implements the redaction function at the message level, ensuring fine-grained redaction. Furthermore, redaction transactions are accepted only if they possess a valid digital signature from the transaction sender or a TA, and an *ATS-IQ* signature from δ TAs. This design addresses the single point of failure risk associated with compromised TAs. Neither of the aforementioned security failure modes affects other operations of the blockchain system, as full nodes are not required to wait for redaction approval before mining other transactions.

6.4.3 G3 (Controlled Delete-only Redaction)

When data is redacted in our proposed protocol, it is replaced only by the block and transaction index of the redaction transaction, which must contain valid redaction signatures. Hence, even if a majority ($\geq \delta$) of the TAs are compromised, the redaction function will still prevent any arbitrary data tampering. This is a key advantage over state-of-the-art chameleon hash approaches, where a similar security failure could allow an adversary to arbitrarily tamper with the transaction data.

6.4.4 G4 (Accountability)

Every data redaction order requires a redaction transaction that must include the public keys and signatures of the entities that issue and approve the order. This design ensures accountability of redaction, even if most ($\geq \delta$) of the TAs are compromised. Since our approach is built for consortium blockchains, which require identity verification, honest TAs and auditors can determine the identities of any malicious adversaries associated with compromised public keys.

7 Proof-of-Concept Implementation

7.1 Experiment Setup

We implement a modified consortium blockchain system with fundamental blockchain operations that integrate the proposed redaction functions using Python 3.7 and ECDSA for the digital signature scheme to demonstrate our proposal. We performed the experiment in the following environment: Ubuntu 22.04, Intel(R) Core(TM) i5-11400H CPU @ 2.70 GHz, 8 GB RAM, and 512 GB SSD storage.

Data	**Size (Byte)**	
ATS-IQ	$\delta = 26$	$\delta = 101$
Signing	4,468	17,359
Combining	519	1,820
Verifying and Tracing		2
Total Communication	**4,989**	**19,181**
Redacted *mtx.d*		32 + 4
RTX Transaction Hash		32
$ri := \langle mb_{mtx_i}, ti_{mtx_i} \rangle$		(32 + 4)**len*(*ri*)
$rs := \langle pk_{rq}, \sigma \rangle$		33 + 72
$taa := \langle \sigma_m, e, Si \rangle$	519 + 32 + 52	1,820 + 32 + 202
Total Storage	**776 + (36**i*)**	**2,227 + (36**i*)**

Table 2: Redaction Data Overhead

7.2 Storage Overhead Evaluation

The storage cost for each redaction operation, which requires mining of one RTX and replacement of the *d* field in each redacted TX with $\langle mb_{rtx}, ti_{rtx} \rangle$ has a storage cost of 36 bytes (assuming the values are 32-byte uint256 and 2-byte uint16 data types, respectively). The RTX itself requires a storage of 32 bytes for the transaction hash, a linear increase of 36 bytes from $\langle mb_{mtx}, ti_{mtx} \rangle$ per transaction in *ri*, 105 bytes for the public key and signature pair included in *rs*, 32 bytes for the hash of *PK*, and *Si* is stored in a 2-byte uint16 value for each signer. The storage cost of the 2-byte public key index value is significantly lower than the standard 32-byte Schnorr ATS public key.

Redactable Mobility Message Package In this scenario, the redaction functions are implemented for smart mobility-related data. Using the European Telecommunications Standards Institute (ETSI)'s standard setting for traffic data message relaying, the package data size is set to 800 bytes (ETSI, 2012). The 800-byte message size allows $\delta = 26$ redactions for 1 TX with small storage overhead (776 + 36 = 812 bytes). However, each redaction operation is required to be performed on at least 3 TXs (2,400 bytes) for $\delta = 101$ to produce no storage overhead in this scenario (2,227 + 108 = 2,335 bytes).

Redactable Personal Data For this scenario, the redaction functions are implemented only for personal data. As there is no standard for personal data sharing, we assume that the data can be of arbitrary size, and the number of transactions to store this data is significantly fewer than transactions for smart mobility-related data. We use the common largest mobility data package at 2,048 bytes (Lopes et al., 2019) for this evaluation. The 2,048-byte mes-

TX Type	TX Generation		TX Validation		TX Redaction	
	Time (ms)	Overhead	Time (ms)	Overhead	Time (ms)	Overhead
Γ.TX	8.45	baseline	5.56	baseline	-	-
Γ^R_δ.TX	8.49	0%	5.63	1%	-	-
Γ^R_δ.RTX $len(ri) = 1$	8.17	0%	5.19	0%	-	-
Γ^R_δ.RTX $\delta = 101$, $len(ri) = 1$	32.58	286%	7.26	31%	8.15	baseline
Γ^R_δ.RTX $\delta = 101$, $len(ri) = 100$	32.87	289%	10.20	84%	12.56	54%
Γ^R_δ.Redacted TX $\delta = 101$	-	-	7.14	28%	-	-
Multi-Authority Chameleon hash TX* $\delta = 101$	702.56	8,233%	23.71	328%	462.67	5,590%

Table 3: Performance Overhead

sage size allows $\delta = 101$ redactions for 1 TX to be performed with a small additional storage overhead (2,227 + 36 = 2,263 bytes, which is 10% overhead of 2,048 bytes).

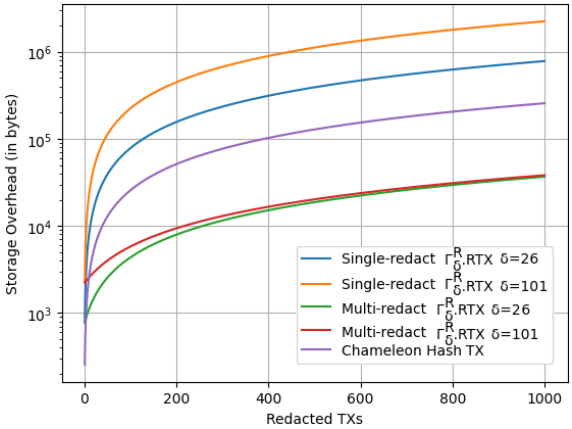


Figure 1: Multi-message Redaction Storage Overhead

As shown in the above evaluation, the storage requirement of our proposed approach increases linearly along with the value δ. Therefore, a higher δ parameter can impact the scalability of the protocol more, compared to the 256-byte chameleon randomness value storage requirement for each transaction in the state-of-the-art (Ateniese et al., 2017). Nevertheless, these overheads can be offset in the overall performance, as our protocol allows multiple transaction redactions per RTX, as illustrated in Figure 1.

7.3 Performance Evaluation

The results of the performance experiment are shown in the average of 1,000 tests timed using the Python *timeit* module. The control groups to evaluate the effect that our proposed redaction protocol has on the blockchain performance are immutable blockchain transactions ($\Gamma.TX$). Both $\Gamma.TX$ and $\Gamma^R_\delta.TX$ contain 800 bytes of random data in d.

As shown in Table 3, the results indicate that our Γ^R_δ protocol yields a similar performance to the immutable blockchain Γ for normal blockchain operations, while the multi-authority chameleon hash primitive with the same δ parameter used in state-of-the-art studies (Tian et al., 2024; Yang et al., 2025) will affect the blockchain performance for any mutable transaction. Meanwhile, the moderate additional performance overhead from $\Gamma^R_\delta.RTX$ for both single- and multi-message redaction still demonstrates a significant improvement compared to the state-of-the-art approach. As such, the results demonstrate superior delete-only enforcement security and performance of our proposed approach over the state-of-the-art.

Our redactable blockchain protocol Γ^R_δ does not directly affect the blockchain consensus delay for two reasons. First, the modification of the blockchain protocol in Γ^R_δ does not change the inheritance consensus and network mechanism of the immutable blockchain Γ protocol; our protocol is implementable as is to all available consensus mechanisms for consortium blockchains. Second, full nodes can perform validation and mining of other transactions while waiting for redaction transactions to gain TAs' approval. As full nodes do not have to recompute the mining proof to perform the redaction, the consensus and network delay are instead implicitly incurred from the storage overhead (Table 2) and transaction operation overhead (Table 3).

8 Conclusion and Future Work

We introduce a redactable consortium blockchain design incorporating a novel multi-message-level data redaction approach. Authentication is secured through the integration of cryptographic digital signatures, accountable threshold signatures, and meta-transactions. This redaction mechanism offers fine-grained access control, enabling transaction senders or TAs to initiate and support large-scale multi-message redaction operations. To ensure accountability, the original transaction message is strictly replaced by an index reference to the redaction transaction, which includes the redaction issuers' digital signature and approvers' threshold signature. This design effectively confines the operation to a controlled delete-only function. Our proof-of-concept evaluation demonstrates that the redaction functions in-

cur minimal performance overhead relative to current state-of-the-art, thereby establishing the protocol's viability for smart mobility. Despite its advantages, the proposed model is still limited by its reliance on trusted authorities and substantial storage overhead for redaction signature proof. Future work could investigate fully decentralised approaches (e.g., public blockchain) and alternative accountable threshold signature schemes that can mitigate storage overhead.

ACKNOWLEDGEMENTS

This work was supported by a project funded by the Engineering and Physical Sciences Research Council (EPSRC) under Grant EP/Y002946/1

REFERENCES

Al-Zaben, N., Hassan Onik, M. M., Yang, J., Lee, N.-Y., and Kim, C.-S. (2018). General data protection regulation complied blockchain architecture for personally identifiable information management. In *2018 International Conference on Computing, Electronics & Communications Engineering (iCCECE)*, pages 77–82, Southend, UK. IEEE.

Ateniese, G., Magri, B., Venturi, D., and Andrade, E. (2017). Redactable blockchain – or – rewriting history in bitcoin and friends. In *2017 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 111–126, Paris, France. IEEE.

Boneh, D. and Komlo, C. (2022). Threshold signatures with private accountability. In Dodis, Y. and Shrimpton, T., editors, *Advances in Cryptology – CRYPTO 2022*, pages 551–581, Cham. Springer Nature Switzerland.

Cai, X., Ren, Y., and Zhang, X. (2020). Privacy-protected deletable blockchain. *IEEE Access*, 8:6060–6070.

Chen, X. and Gao, Y. (2022). Cdedit: Redactable blockchain with cross-audit and diversity editing. In *2022 IEEE International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, pages 945–952, Wuhan, China. IEEE.

Consulting, I. (2018). General data protection regulation (gdpr) – final text neatly arranged. [Accessed: 2024-06-26].

Cui, M., Han, D., Liu, H., Li, K.-C., Tang, M., Chang, C.-C., Ayaz, F., Sheng, Z., and Guan, Y. L. (2024). Secure data sharing for consortium blockchain-enabled vehicular social networks. *IEEE Transactions on Vehicular Technology*, 73(12):19682–19695.

DELPHI project (2025). Delphi federated network of platforms for passenger and freight intermodality. [Accessed: 2025-07-17].

Dorri, A., Kanhere, S. S., and Jurdak, R. (2019). Mofbc: A memory optimized and flexible blockchain for large scale networks. *Future Generation Computer Systems*, 92:357–373.

Duan, P., Wang, J., Zhang, Y., Ma, Z., and Luo, S. (2023). Policy-based chameleon hash with black-box traceability for redactable blockchain in iot. *Electronics*, 12(7):17.

ETSI (2012). Etsi tr 102 861 v1.1.1 (2012-01). [Accessed: 2024-11-15].

Fan, Q., Xin, Y., Jia, B., Zhang, Y., and Wang, P. (2023). Cobats: A novel consortium blockchain-based trust model for data sharing in vehicular networks. *IEEE Transactions on Intelligent Transportation Systems*, 24(11):12255–12271.

Garay, J., Kiayias, A., and Leonardos, N. (2015). The bitcoin backbone protocol: Analysis and applications. In Oswald, E. and Fischlin, M., editors, *Advances in Cryptology - EUROCRYPT 2015*, pages 281–310, Berlin, Heidelberg. Springer Berlin Heidelberg.

Heo, J. W., Ramachandran, G., and Jurdak, R. (2024). Decentralised redactable blockchain: A privacy-preserving approach to addressing identity tracing challenges. In *2024 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, pages 215–219, Dublin, Ireland. IEEE.

Jia, M., Chen, J., He, K., Du, R., Zheng, L., Lai, M., Wang, D., and Liu, F. (2022). Redactable blockchain from decentralized chameleon hash functions. *IEEE Transactions on Information Forensics and Security*, 17:2771–2783.

Li, X. and Ma, C. (2021). Redactable blockchain based on unforgeable signatures for supporting fast verification. In *2021 IEEE 20th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, pages 1202–1207, Shenyang, China. IEEE.

Li, X., Xu, J., Yin, L., Lu, Y., Tang, Q., and Zhang, Z. (2023). Escaping from consensus: Instantly redactable blockchain protocols in permissionless setting. *IEEE Transactions on Dependable and Secure Computing*, 20(5):3699–3715.

Li, Y., Sengupta, B., Tian, Y., Yuan, J., and Yuen, T. H. (2024). Message control for blockchain rewriting. *IEEE Transactions on Dependable and Secure Computing*, PP:1–14.

Lopes, L. H. S., Mini, R. A. F., and Cunha, F. (2019). A v2x approach for data dissemination in vehicular ad hoc networks. In *2019 IEEE Symposium on Computers and Communications (ISCC)*, pages 1–6, Barcelona, Spain. IEEE.

Lu, Y., Zhang, J., Qi, Y., Qi, S., Zheng, Y., Liu, Y., Song, H., and Wei, W. (2022). Accelerating at the edge: A storage-elastic blockchain for latency-sensitive vehicular edge computing. *IEEE Transactions on Intelligent Transportation Systems*, 23(8):11862–11876.

Ma, J., Xu, S., Ning, J., Huang, X., and Deng, R. H. (2022). Redactable blockchain in decentralized setting. *IEEE Transactions on Information Forensics and Security*, 17:1227–1242.

Micali, S., Ohta, K., and Reyzin, L. (2001). Accountable-subgroup multisignatures: extended abstract. In *Proceedings of the 8th ACM Conference on Computer and Communications Security*, CCS '01, page 245–254,

New York, NY, USA. Association for Computing Machinery.

MOBI (2025). Mobility open blockchain initiative. [Accessed: 2025-07-18].

Ou, C. and Ye, G. (2023). Collusion-resistant editable blockchain with punishment for malicious behavior using chameleon hash algorithm. In *2023 International Conference on Computers, Information Processing and Advanced Education (CIPAE)*, pages 540–545, Ottawa, ON, Canada. IEEE.

Politou, E., Casino, F., Alepis, E., and Patsakis, C. (2021). Blockchain mutability: Challenges and proposed solutions. *IEEE Transactions on Emerging Topics in Computing*, 9(4):1972–1986.

Puddu, I., Dmitrienko, A., and Capkun, S. (2017). μchain: How to forget without hard forks. *IACR Cryptol. ePrint Arch.*, 2017:106.

Shao, W., Wang, J., Wang, L., Jia, C., Xu, S., and Zhang, S. (2023). Auditable blockchain rewriting in permissioned setting with mandatory revocability for iot. *IEEE Internet of Things Journal*, 10(24):21322–21336.

Shen, J., Chen, X., Liu, Z., and Susilo, W. (2023). Verifiable and redactable blockchains with fully editing operations. *IEEE Transactions on Information Forensics and Security*, 18:3787–3802.

Tian, Y., Liu, B., Li, Y., Szalachowski, P., and Zhou, J. (2024). Accountable fine-grained blockchain rewriting in the permissionless setting. *IEEE Transactions on Information Forensics and Security*, 19:1756–1766.

Wei, L., Zhang, Y., Cui, J., Zhong, H., Bolodurina, I., and He, D. (2024). A threshold-based full-decentralized authentication and key agreement scheme for vanets powered by consortium blockchain. *IEEE Transactions on Mobile Computing*, 23(12):12505–12521.

Wu, Z., Wang, L., Zhang, X., and Feng, X. (2025). Emt: Extended merkle tree structure for inserted data redaction in permissioned blockchain. *IEEE Transactions on Network Science and Engineering*, 12(4):3025–3038.

Yadav, S., Singh, K., Yadav, A. K., Shariq, M., Chaudhry, S. A., Das, A. K., and Manjul, M. (2025). Efficient and reliable information sharing for internet of vehicles using trust and blockchain. *IEEE Transactions on Vehicular Technology*, pages 1–13.

Yang, Y., Chen, Y., Liu, Z., Tan, C., and Luo, Y. (2025). Verifiable and redactable blockchain for internet of vehicles data sharing. *IEEE Internet of Things Journal*, 12(4):4249–4261.

Ye, T., Luo, M., Yang, Y., Choo, K.-K. R., and He, D. (2023). A survey on redactable blockchain: Challenges and opportunities. *IEEE Transactions on Network Science and Engineering*, 10(3):1669–1683.

Zhang, D., Le, J., Lei, X., Xiang, T., and Liao, X. (2024). Secure redactable blockchain with dynamic support. *IEEE Transactions on Dependable and Secure Computing*, 21(2):717–731.

Zhou, J., Hong, J., Zhu, C., Zou, F., and Hua, C. (2024). Trustful blockchain-enabled identity management for vanet with short-latency authentication. In *2024 International Conference on Computing, Networking and Communications (ICNC)*, pages 659–663, Big Island, HI, USA. IEEE.